



redis

REDIS → Remote Dictionary Server

- 1) opensource, in-memory data store
- 2) Support key-value and various data structure
- 3) versatile -
 - Caches
 - Distributed locks
 - Leaderboard
 - Kafka
 - Rate limiter
- 4) Written in C for high performance and low latency.

Why Redis is so fast?

1. Everything is happening in RAM

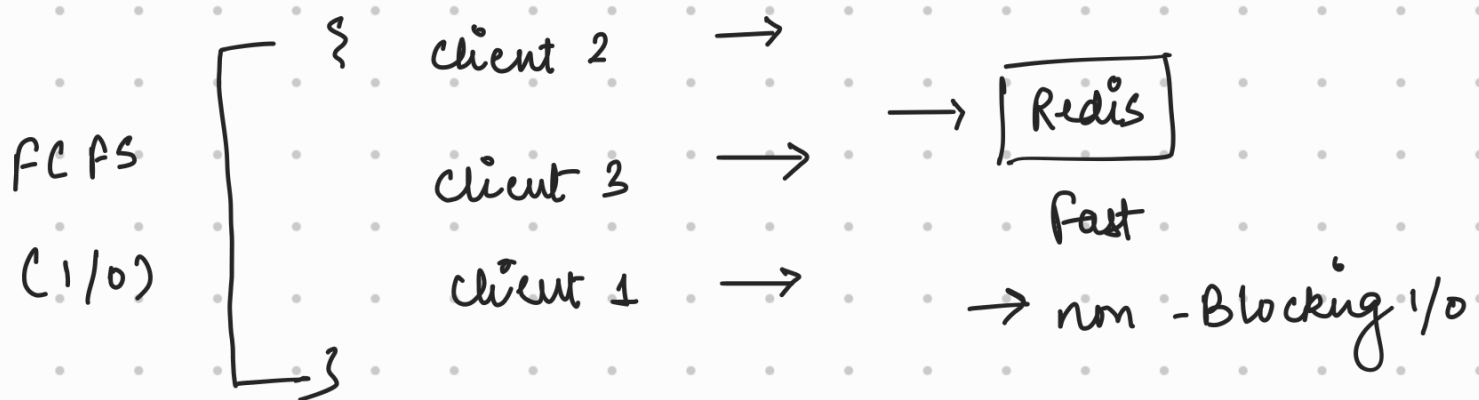
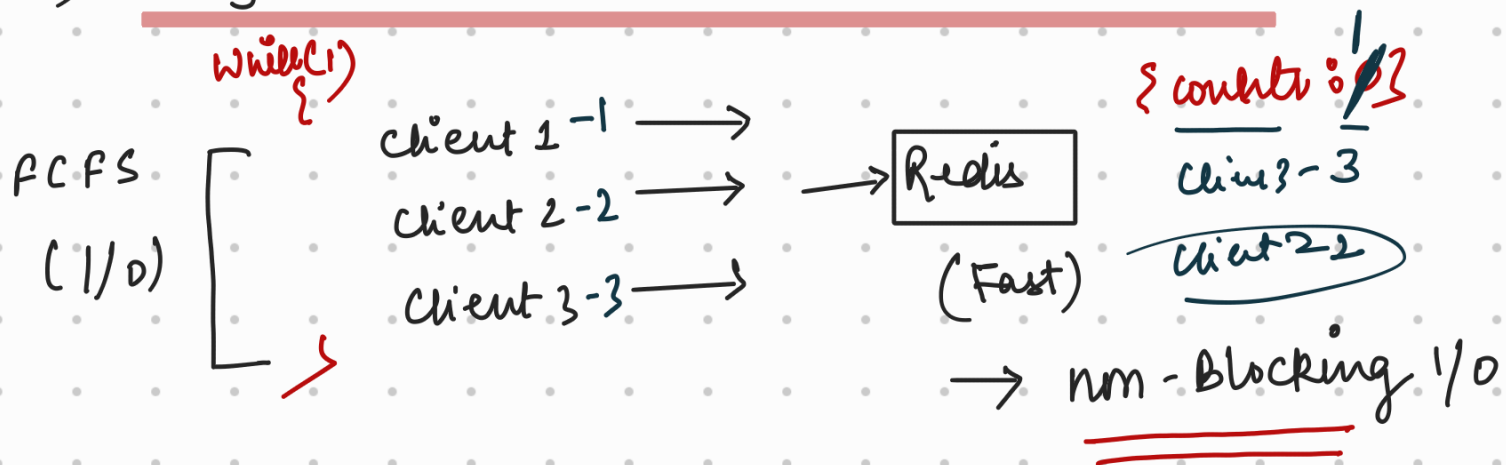
Redis (RAM)	100-(nanosec)	RAM
SSD (flash memory)	100-(microsec)	databases
HDD (Rotational Disk)	5-10 (millisec)	legacy systems

10M+ iops

~ 100-500K ops

100-200 iops

2.) Single thread - Event Driven Architecture



→ let's say if you hit
a Billion / milli → it fail

1) But not possible

2) Here is how our



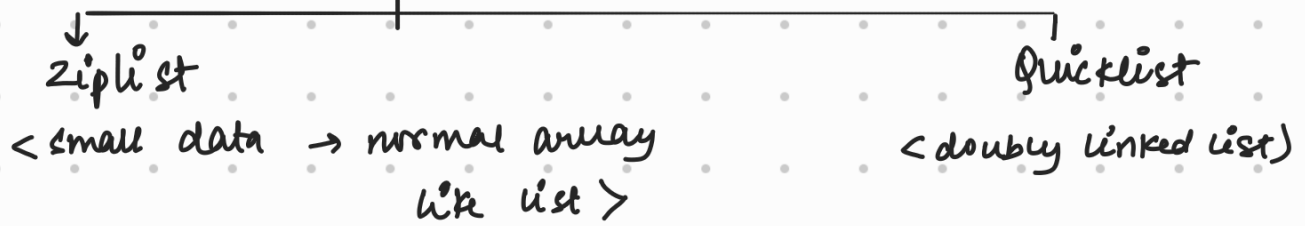
3.) Redis Data Structures

a) String → no change → { "key1" → "hi" }

b) int → no int → { "key2" → 1 }

if { key3 : "123" } → how will redis store it?

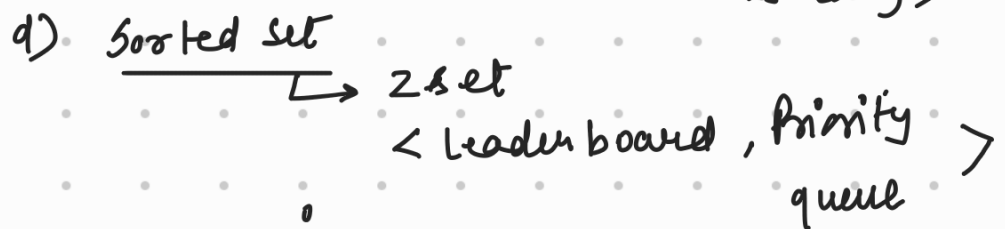
c) lists



d) set



d) Sorted set



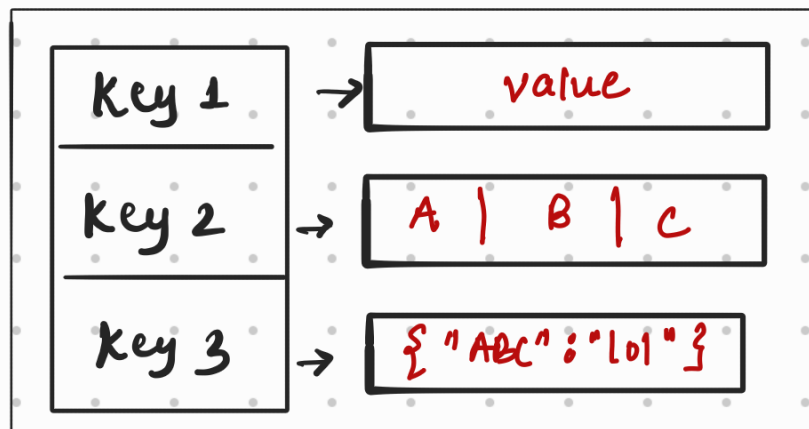
e) streaming

↳ time ordered db in redis

{10000-0: "K"} → you can perform grouping
{10002-1: "P"} → help to act as a kafka
{10003-2: "S"} → history } → purpose.

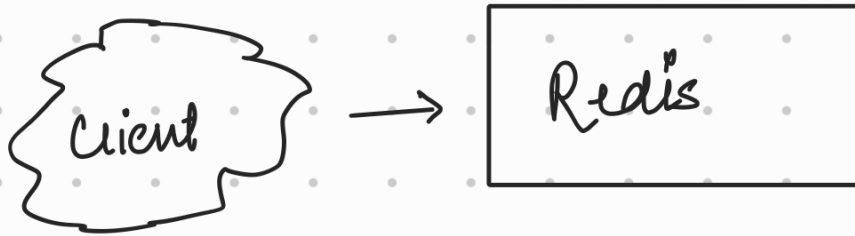
Redis internals

- 1) Single threaded
- 2) In memory
- 3) Data structure Server



Architecture

1) Single Node Architecture



Pros

- 1) Simple to setup
- 2) Small setup, testing, small-scale apps

Cons

- 1) No high availability
- 2) Data loss unless you use →
|

↓
AOF

(Append only file)

→ Log every write operation in Disk

→ Every write (safest)

→ Every second (default)

→ or decide

↓
RDB

(Redis Database Backup)

↳ Data save to

dump.rdb

→ in every 5 minute / 1min

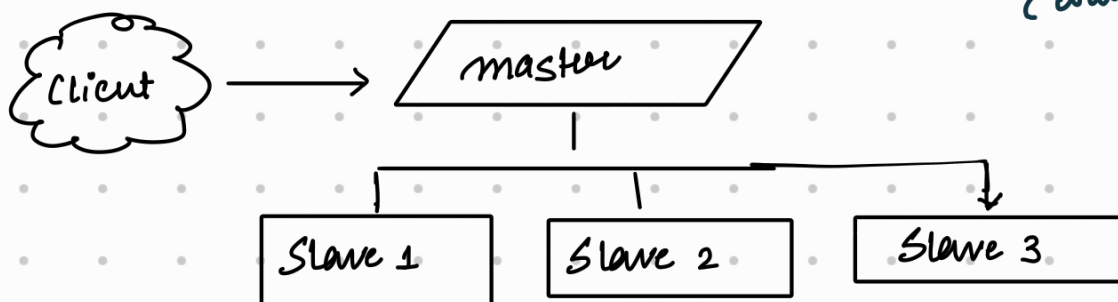
→ Fast

→ maybe lost recent writes

2) Master Slave Architecture

Σ 10

{write: 1}



(*) write at master

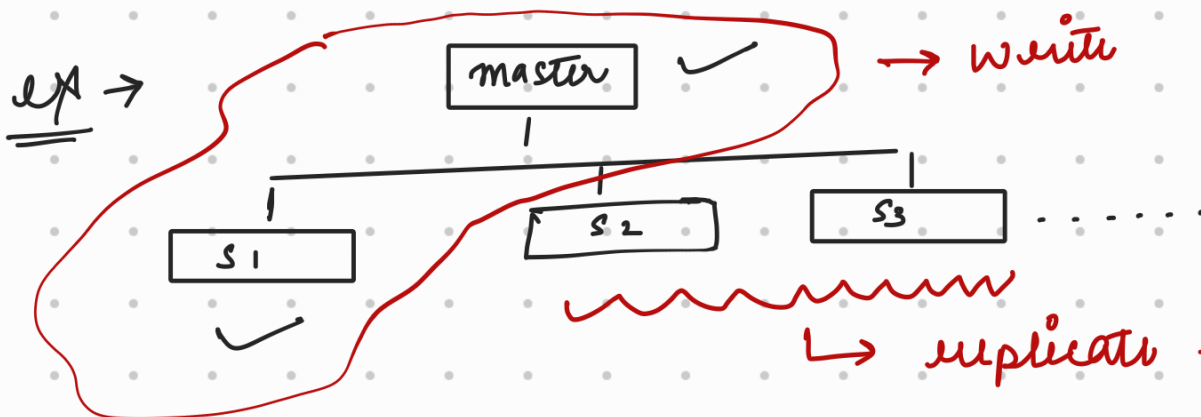
(*) Read at slave → Read >>> write

Why we do this?

1) Because if master goes down we make any slave new master

future →

1) Redis 5 → is coming up with atleast write in x slaves then return

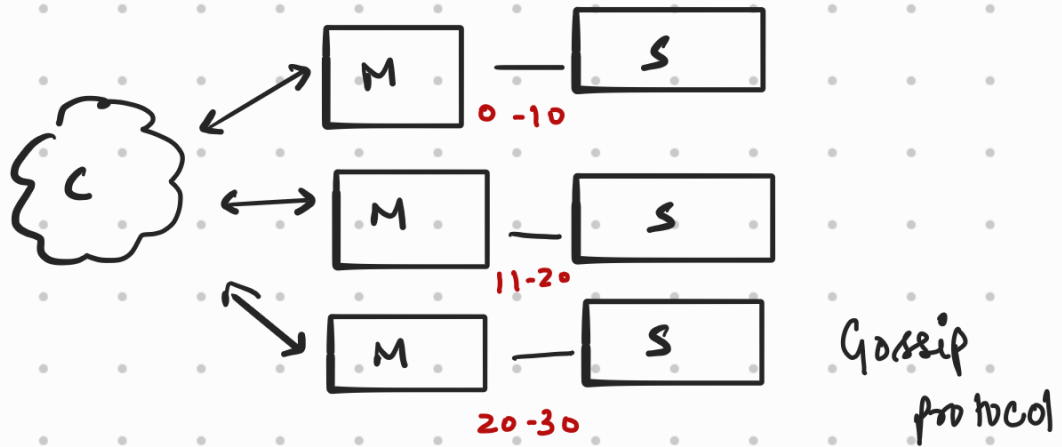


When replication slow Down?

1) too many slaves

2) RDB - full sync (Disconnection) → of slaves

3) Redis Cluster Architecture



Internally

1) Redis make slots $\rightarrow 16384$

Suppose 3 master

M_1

0 - 54630

M_2

54631 - 10912

M_3

10913

- 106384

"apple"

$\rightarrow \text{hash}(\text{Key}) \% 16384 \rightarrow \text{Suppose}$
 $\rightarrow (8000) \rightarrow \text{node}(m_2)$

$\rightarrow$ Data will be stored in key-value.

Confusion

$M = 24$

8

X

8

8

0-8

$\rightarrow$ rebalancing
 8-16 11-24

$M = 24$

0-6

6

6-12

6

12-18

6

② $\rightarrow$ 0

6

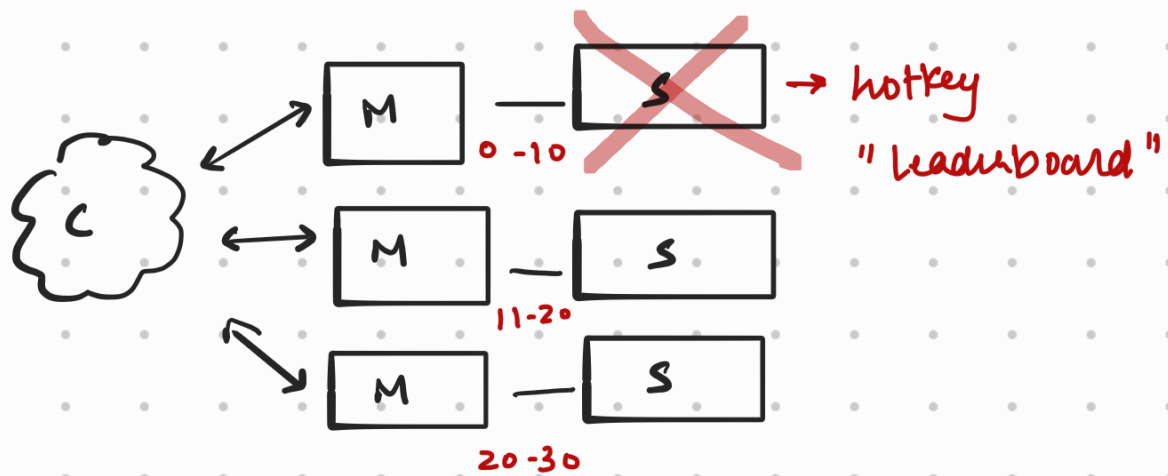
$\rightarrow$ Smart Client

$\rightarrow$ Java $\rightarrow$ Jedis

$\rightarrow$ node.js $\rightarrow$ ioredis

$\rightarrow$ python $\rightarrow$ redis-py-cluster

Hot-key Problem in Redis



Solution

key → "movie:global" → "not key"

↓ let's not save it like this

"movie:global:1"

→ node 1

"movie:global:2"

→ node 2

↳ now traffic is divided

→ But redis don't do it automatically it is done by the help of code.

→ Redis don't shard internally → it can be considered as a flaw.

Expiration in Redis

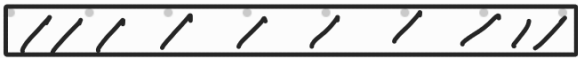
↳ TTL (Time to live on key)

SET user:10 "24" EX 10
 ^{~~~~~}
 { expire after 10 seconds

how it works

- Redis run a cron to delete all the nodes that has (TTL) in every 100 ms
- But pick only (20 key per cycle)
- Depends on CPU usage as well

Redis Eviction Policy

 — Full
↙ how to create some space?

1) No Eviction?

→ Give error there is no space

2) All keys LRU?

→ delete all the keys that are (least Recently used).

3) Volatile - LRU →

→ delete all the keys with TTL
→ with the concept of LRU.

4) allkeys - random →

→ delete random keys

5) volatile - random →

→ delete randomly from all keys

6) volatile - TTL →

→ delete the keys that have
Shortest (TTL).

Ques

Example → 10 Kb

→ and 10 records of 1 Kb each

→ if 1 more record come how many
keys will be deleted in Eviction?

1) Redis as Key-value Store

Commands:-

```
SET user:1001 "Savage"  
GET user:1001
```

2) Redis as Distributed Locks

```
SET user:lock:10 "value" NX EX 10
```

only set if not

Expire after (n) sec

(*) according to your use case.

3) Redis as Rate Limiter

(30) apis 20sec

```
SET user:Rate:1 1 EX 120
```

```
INC user:Rate:1 2
```

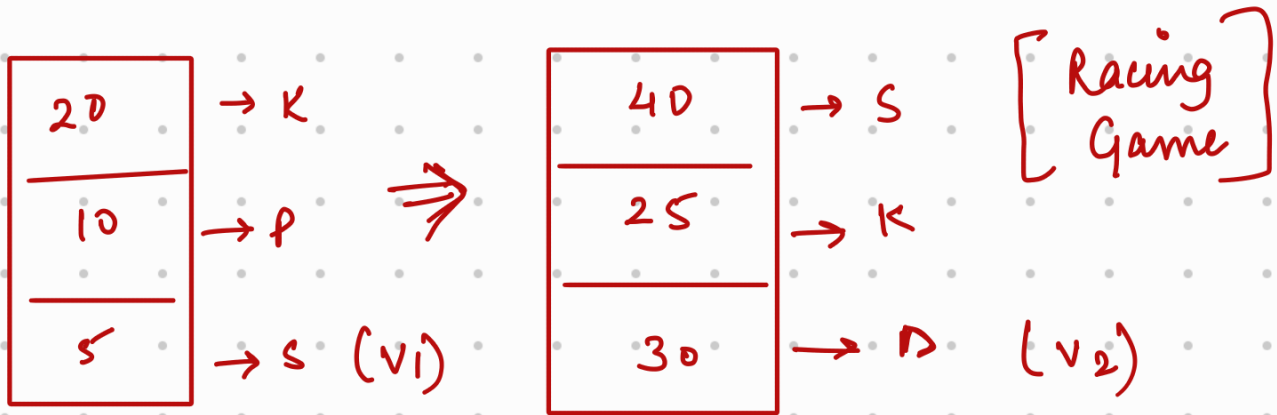
```
INC user:Rate:1 3
```

~~#~~ ~~#~~ ~~#~~

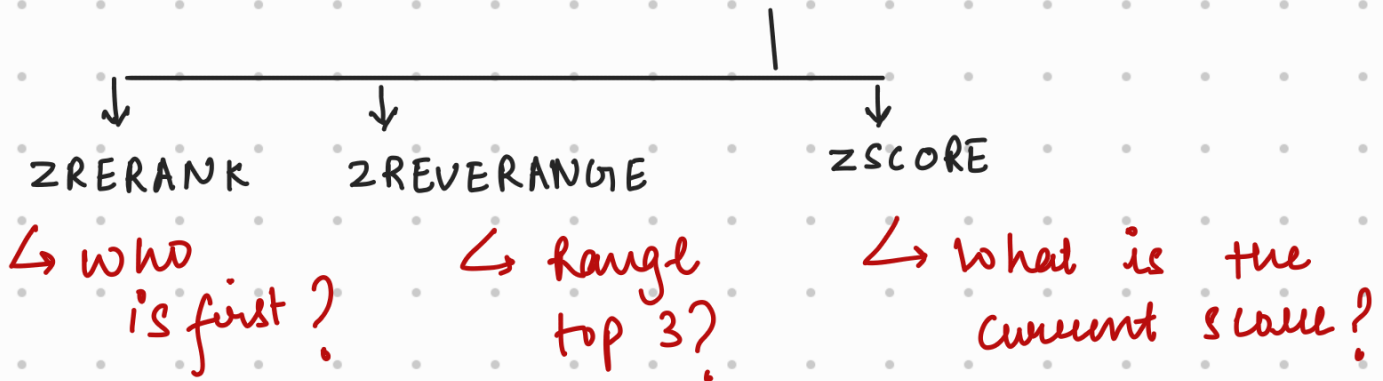
∴ → Get "user:Rate:1"

→ if it reaches
above threshold
→ mark it failed

4) Redis as a leaderboard



```
ZADD game:score 300 "Alice"  
ZADD game:score 500 "Bob"  
ZADD game:score 800 "Savage"
```



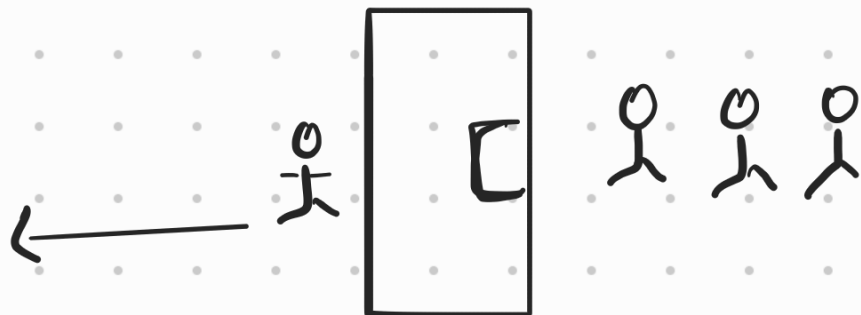
Redis as Kafka

Consumer

FOOD

Consumer 2

DRINK



Producer
two consumer group

food

drink

- XADD order * type food id 1 item Burger 2
- XADD order * type drink id 2 item Coke 1
- XADD order * type food id 3 item Burger 1
- XADD order * type drink id 4 item Coke 2

DRINK

→ XREADGROUP GROUP beverage
Consumer 10 streams count >

FOOD

→ XREADGROUP GROUP food
Consumer 10 streams count >

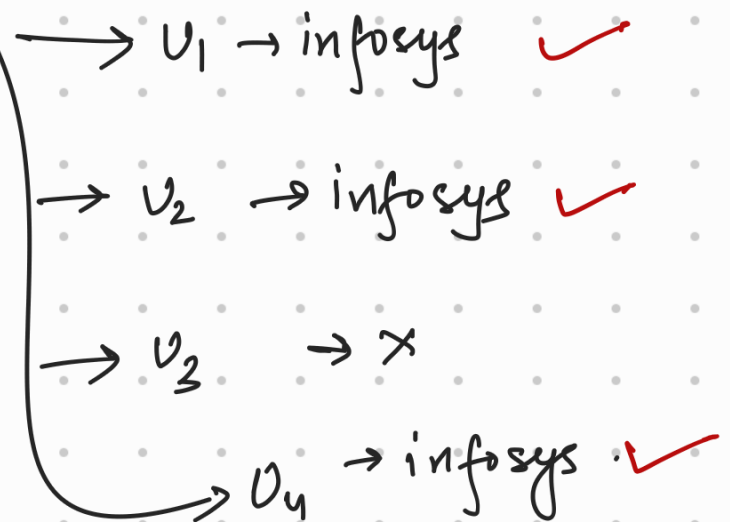
Redis as Pub/sub

Stock
Service

Publish



1) send notification to
everyone who have
infosys



Publish infosys:alert "infosys crash" → Stock Service

Subscribe infosys:alert → Notification Service



Redis as Geolocation Store

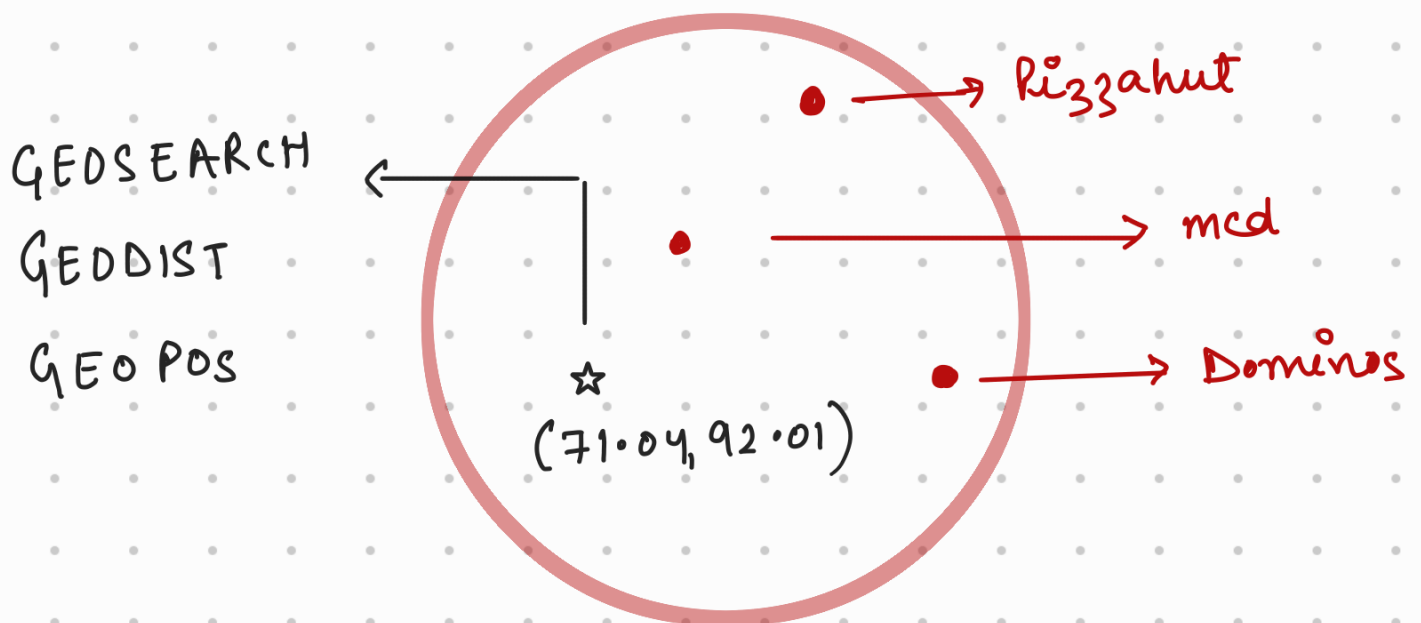
Format of command: -

→ GEOADD LAT LONG NAME

ex GEOADD 77.884 99.001 "Pizzahut: Delhi"

GEOADD 77.05 99.02 "Domino: Delhi"

GEOADD 77.09 99.05 "mcd - : Delhi"

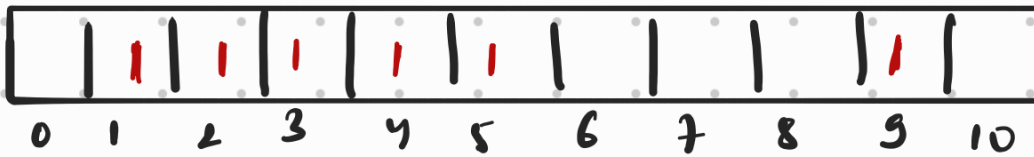


Real life

→ find cab under 1 km

→ all the nearby cafes (10 km radius)

Redis as Bloom Filter



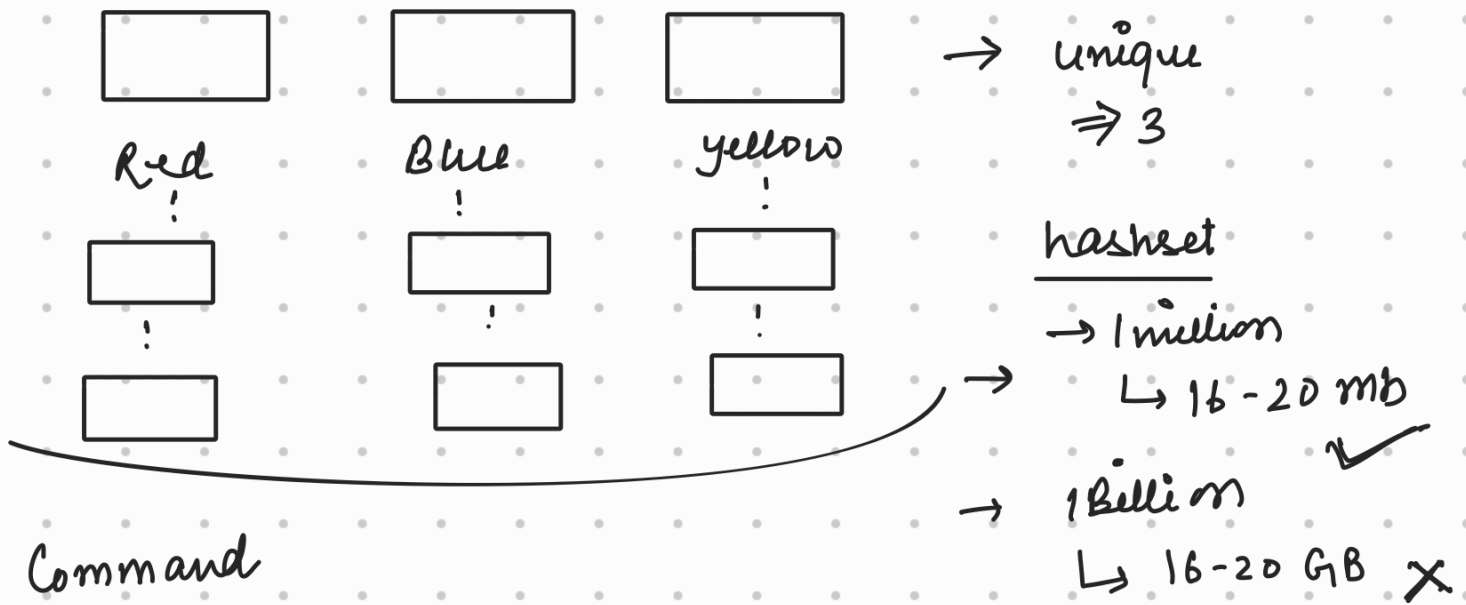
	Savage	Kp	Dant	→ insta
Hash 1	1	3	5	
Hash 2	2	4	9	

	"Salman Bhai"	"Srk"	"Shushant"
Hash 1	3	0	8
Hash 2	9	9	10

{ → "we cannot not create" (False Positive)
 { → "we will make user" (True Positive)

BF·ADD	username	Savage	} → Adding	
BF·ADD	username	KP		
BF·ADD	username	Dant		
BF·EXIST	username	SalmanBhai	X	} → Searching
BF·EXIST	username	Srk	✓	
BF·EXIST	username	Shushant	✓	

Redis as hyperloglog log



Command

PFADD visitor "U1"
PFADD visitor "U2"
PFADD visitor "U3"
PFADD visitor "U1" → duplicate
→ (3)

hyperloglog → (0.81%) → error

→ used in Big Data and IoT where hash-map fails.

→ take only 12 kb space